LedgerForge — Phase 1 Report

WorkforceOS: Core Payroll, Accounting & Employee Data Engine

Candidate Name: Ancillin P G | **Track:** Python Developer
Phase: Phase 1: Foundation (Days 1 to 5 / Tasks LF-01 to LF-05) | **Git Tags:** phase-1, v0.0.1
GitHub Repository: https://github.com/Ancilin/Foundation-Project-Setup-Employee-Data-Model

---

1. Phase 1 Task Execution & Evidence Table (Days 1 to 5)

Day / Task	Objective & Scope	Status	Deliverables & Evidence Links
Day 1 **LF-01**	**Workspace & First API** FastAPI workspace, /docs, /health, GET /departments (11 depts), pytest suite, 10-step README, tag v0.0.1.	**DONE**	• Docs & Health: /docs, /health • Router: app/api/routes/departments.py • Run Guide: README.md (10 steps) • Tags: v0.0.1, phase-1
Day 2 **LF-02**	**Employee & Org Data Model** SQLAlchemy models: Department, Role, Employee, EmployeeAssignment. Self-referencing manager_id, indexes, Alembic, ER diagram.	**DONE**	• ER Diagram: docs/er-diagram.md • Models: app/models/ (employee, assignment) • Migrations: migrations/versions/ (0001, 0002, 0003) • 3-line nullability rationale & Dept D05 query
Day 3 **LF-03**	**One-Command Data Seeder** Repeatable data seeder: 11 depts, 58 roles, 745 employees, 1,606 assignments, 14 components, 12,295 structures, 2,459 revisions.	**DONE**	• Seed Script: python -m scripts.seed --reset • Official Verifier: scripts/verify_ledgerforge.py • Idempotent execution, zero foreign key drift • 5/5 official integrity checks passed
Day 4 **LF-04**	**Employee & Team APIs** CRUD on /employees, name/email search, dept filter, pagination, soft delete (exit_date), manager direct reports (/team).	**DONE**	• Endpoints: app/api/routes/employees.py • Team: GET /employees/{id}/team • 26 API tests covering 404, 422, pagination • Pydantic v2 schemas and validation
Day 5 **LF-05**	**Salary Structures & Revisions** Components, effective-dated structures (effective_from/to), salary revisions, Decimal math, get_structure, revise_salary, >20% raise flag.	**DONE**	• Models: app/models/salary.py (Decimal) • Service: app/services/salary_service.py • Tests: tests/test_salary_structures.py • Boundary tests across 3 employees passed

2. Proof It Works: Automated Tests & Verification Output

Automated Test Suite: 71 passed, 0 failed (100% Pass Rate in 64s)
Complete test suite runs cleanly across SQLite and PostgreSQL engines.

```
============================= test session starts ==============================
platform win32 -- Python 3.12.10, pytest-8.4.2
rootdir: C:\Users\ancillin\Desktop\project task
collected 71 items

tests/test_health.py (3 tests: /health, /api/v1/health, /docs) ........... PASSED
tests/test_constraints.py (30 tests: unique email, enums, dates, bands) . PASSED
tests/test_hierarchy.py (7 tests: manager links, cycles, reports) ........ PASSED
tests/test_migrations.py (3 tests: upgrade head, rollback base, no-drift) PASSED
tests/test_api_employees.py (18 tests: CRUD, search, filter, pagination) . PASSED
tests/test_api_departments.py (4 tests: list, department team query) ..... PASSED
tests/test_salary_structures.py (3 tests: get_structure, revisions) ...... PASSED
tests/test_seed_and_export.py (3 tests: seed pipeline & CSV exports) ..... PASSED

======================== 71 passed, 4 warnings in 64.16s =======================
```

Official Integrity Verifier Output (verify_ledgerforge.py)

```
PASS Every journal entry balances - debits-credits=0.0, unbalanced entries=0
PASS Payroll equals ledger, every month - largest monthly difference=0.00
PASS Receivables equal open invoices - ledger 1100=139,394,278.00, open invoices=139,394,278.00
PASS Payroll employees exist - payroll rows with unknown employee=0
PASS Nobody is paid after leaving - payroll rows after exit_date=0

5/5 checks passed cleanly
```

Database & Master Dataset Record Counts

Entity / Table	Dataset Target	Database Count	Verification Details
Total Employees	745	**745**	441 Active, 304 Resigned/Terminated (Workbook 01)
Departments	11	**11**	D00 (Executive) through D10 (Legal & Compliance)
Roles & Salary Bands	58	**58**	Job levels 1–7 with band min, mid, and max bounds
Employee Assignments	1,606	**1,606**	Department, role, and manager change history (LF-02)
Salary Components	14	**14**	BASIC, HRA, SPECIAL_ALLOWANCE, PF, ESI, etc. (LF-05)
Salary Revisions	2,459	**2,459**	Effective dates, pct_change, approved_by (LF-05)
Salary Structures	12,295	**12,295**	Decimal(12,2) effective-dated component breakdowns

3. What I Learned (In My Own Words)

Building Phase 1 taught me how professional enterprise backend services are architected from the ground up. I learned the critical value of a clean layered design—strictly isolating route handlers, Pydantic validation schemas, and business service logic so that code remains modular, testable, and free of HTTP/SQL coupling. I gained deep practical experience designing relational data models with SQLAlchemy 2.0 and Alembic migrations, managing self-referential manager hierarchies, enforcing database constraints, querying effective-dated temporal tables, and using Python Decimal to ensure absolute paisa-level precision across all financial compensation structures.

4. Problems Encountered & Technical Solutions

1. Self-Referencing Foreign Key Insertion Ordering

Problem: In a single-pass seed, employees frequently referenced a manager_id whose record had not yet been inserted, triggering foreign key violations.
Solution: Implemented a **two-pass seeding algorithm** in scripts/seed.py: Pass 1 inserts all employees with manager_id=None, and Pass 2 establishes the manager relationships once all primary keys exist.

2. Referential Integrity in Bounded Test Runs

Problem: When seeding a subset of employees (e.g. employee_limit=10 during fast unit tests), loading full assignment and revision CSVs threw foreign key errors for excluded IDs.
Solution: Filtered assignment, revision, and structure rows dynamically against currently active employee IDs in the database, preserving referential integrity across all test scopes.

3. Effective Date Boundary Overlaps in Salary Revisions

Problem: Updating salary revisions without precise timestamp boundaries created overlapping date intervals or missing gaps.
Solution: Built revise_salary in app/services/salary_service.py to automatically close the preceding active structure at effective_from - 1 day before creating new component rows, maintaining continuous compensation history.

4. Multi-Threaded SQLite Concurrency for Fast Local Development

Problem: During local testing without Docker, FastAPI worker threads locked the local SQLite connection, causing endpoint timeouts.
Solution: Configured connect_args={'check_same_thread': False} in app/db/session.py, enabling instant local development alongside full PostgreSQL compatibility.

5. Plan for Next Phase (Phase 2 — Core Features)

Phase 2 turns employee and salary foundations into a running payroll and leave engine (Tasks LF-06 to LF-08). The following roadmap will be executed in accordance with the Task Playbook:

Task 6 (LF-06) — Attendance & Leave Service:

• Model leave types (Casual, Sick, Earned), leave applications, approval/rejection workflows, and leave balances.
• Implement holiday calendar engine: skip weekends and public holidays when counting leave days.
• Attendance summary endpoint: record check-ins/check-outs, handle MISSING_CHECKOUT rows without crashing, and provide monthly summaries verified against 3 benchmark employees.

Task 7 (LF-07) — Payroll Calculation Engine:

• Implement calculate_employee_month() as a pure function matching the answer key inside payroll_items.csv.
• Calculate Loss of Pay (LOP) per calendar day, Overtime (2x rate, 50-hour cap, eligible roles only), and statutory deductions.
• Statutory computations: Provident Fund (12% of basic up to ■15,000 ceiling), ESI, Professional Tax, and progressive TDS.
• Verification: Run all 7,340 employee-months and verify via verify_ledgerforge.py payroll to achieve 99%+ exact matches to the paisa.

Task 8 (LF-08) — Payslip Generator:

• Build GET /payslips/{employee}/{run_id} returning full gross-to-net breakdown.
• Generate clean PDF payslips via ReportLab: company header, earnings, deductions, net pay in words, and Indian currency formatting (■ 1,23,456.00).
• Role-based security: employees access only their own payslip (403 forbidden for others); HR and Admin access all.

6. Submission & Verification Sign-Off

Candidate Signature: Ancillin P G | **Date:** October 19, 2026
Git Remote URL: https://github.com/Ancilin/Foundation-Project-Setup-Employee-Data-Model
Git Tags Verified on GitHub: phase-1 and v0.0.1
Collaborator Access: fobesitech invited for private repository code review.